UTM

UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING (WBL)

TUTORIAL III: AI ALGORITHM

GROUP: 8

PREPARED BY :

NAME	MATRIC NO
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258

PREPARED FOR:

DR. ARYATI BINTI BAKRI

TABLE OF CONTENTS

1.0 What is Image Classification?

Image classification is the task of assigning a predefined label (class) to an input image based on its visual content. It is one of the foundational problems in computer vision and deep learning, with applications spanning medicine, security, agriculture, retail, and autonomous driving.

In a typical image classification pipeline, a model receives a raw image as input (a 2D or 3D matrix of pixel values) and outputs a probability distribution across all possible classes. The class with the highest probability is selected as the predicted label.

1.1 Key Characteristics

- Input: RGB or grayscale image represented as a pixel tensor ($H \times W \times C$)
- Output: A class label from a fixed set (e.g., cat, dog, airplane)
- Approach: Feature extraction followed by classification
- Common datasets: MNIST, CIFAR-10, ImageNet, CIFAR-100

1.2 Traditional vs. Deep Learning Approaches

Traditional machine learning methods relied on hand-crafted features such as HOG (Histogram of Oriented Gradients), SIFT, and LBP (Local Binary Pattern), which were manually engineered by domain experts. These features were then fed into classifiers like SVM or Random Forest.

Deep learning, particularly CNNs, revolutionised this paradigm by automatically learning hierarchical features directly from raw pixel data. This end-to-end learning approach dramatically improved accuracy and generalisation, making deep CNN-based classifiers the dominant approach in modern computer vision.

2.0 What is CNN?

A Convolutional Neural Network (CNN) is a class of deep neural network specifically designed to process structured grid data such as images. CNNs exploit the spatial locality of pixels through local connections and shared weights, making them far more efficient and accurate than fully connected networks for visual data.

CNNs were popularised by LeCun et al. (1998) with LeNet-5 for handwritten digit recognition. Modern architectures such as AlexNet, VGGNet, ResNet, and EfficientNet have pushed the frontier further, achieving superhuman performance on benchmarks like ImageNet.

2.1 CNN Architecture

A standard CNN is composed of three major building blocks:

2.1.1 Convolutional Layer

The convolutional layer is the core computational unit of a CNN. It applies a set of learnable filters (kernels) to the input, producing feature maps that capture local patterns such as edges, textures, and shapes.

- Filter/Kernel: A small matrix (e.g., 3×3) that slides across the input
- Stride: The step size of the sliding filter, larger strides reduce output size
- Padding: Zero-padding can be added to preserve spatial dimensions ('same' padding)
- Feature Map: The output of applying one filter across the entire input

Mathematically, the convolution operation at position (i, j) for filter k is:

```
Output[i,j,k] = Σ Σ Input[i+m, j+n] * Filter[m,n,k] + bias[k]
```

2.1.2 Pooling Layer

Pooling layers reduce the spatial dimensions of feature maps, decreasing computational cost and providing spatial invariance. The most common type is Max Pooling, which takes the maximum value within each pooling window:

- Max Pooling: Retains the most prominent feature in each region
- Average Pooling: Computes the mean, smoother but may lose detail
- Global Average Pooling (GAP): Reduces each feature map to a single value, useful before the classifier

2.1.3 Activation Function (ReLU)

Rectified Linear Unit (ReLU) is applied element-wise after each convolution: f(x) = max(0, x). ReLU introduces non-linearity, enabling the network to learn complex mappings, and avoids the vanishing gradient problem common with sigmoid and tanh functions.

2.1.4 Fully Connected (Dense) Layer

After several convolutional and pooling layers, the feature maps are flattened into a 1D vector and passed through one or more fully connected layers that act as a traditional classifier. The final dense layer uses Softmax activation to output class probabilities.

2.1.5 Dropout

Dropout randomly deactivates a fraction of neurons during training (e.g., 50%). This regularisation technique prevents the network from co-adapting too strongly to training data, thereby reducing overfitting.

2.2 Summary of CNN Layers

Table 2.0 CNN Layers

Layer Type	**Purpose**	**Learnable Parameters**
Convolutional	Extract spatial features using learnable filters	Feature maps, bias, kernel size, stride, padding
ReLU	Introduce non-linearity	None (element-wise operation)
Pooling	Down-sample feature maps	Pool size, stride, type (max/avg)
Flatten	Convert 3D maps to 1D vector	None
Dense	Classification based on learned features	Weights, bias, activation
Dropout	Regularisation via random neuron deactivation	Rate (0–1)

3.0 Evaluation of CNN

Evaluating a CNN model requires both quantitative metrics and visual diagnostics to fully understand its performance and identify potential shortcomings.

3.1 Key Evaluation Metrics

- Accuracy: Proportion of correctly classified samples: (TP + TN) / Total
- Loss (Cross-Entropy): Measures how far the predicted probability distribution is from the true label. Lower is better.
- Precision: Of all positive predictions, how many are truly positive: TP / (TP + FP)
- Recall (Sensitivity): Of all actual positives, how many were correctly detected: TP / (TP + FN)
- F1-Score: Harmonic mean of Precision and Recall: 2 × (Precision × Recall) / (Precision + Recall)
- Confusion Matrix: A N×N table showing correct and incorrect predictions per class
- ROC-AUC: Area Under the Receiver Operating Characteristic Curve measures class separability

3.2 Training vs. Validation Curves

Plotting training and validation accuracy/loss over epochs is essential for diagnosing model behaviour. Typical patterns include:

- Good fit: Both curves converge to low loss and high accuracy
- Overfitting: Training accuracy >> validation accuracy; validation loss increases after a point
- Underfitting: Both training and validation accuracy remain low means that model is too simple

4.0 Steps hand on in Python

This section is the practical workflow used to build the image classifier in Python, by using the Tensorflow/Keras library and the CIFAR-10 dataset. The notebooks follow a standard deep learning pipeline where prepared the dataset, establish a simple baseline, build a Convolutional Neural Network (CNN), train it, and finally evaluate their predictions. Each step below corresponds to a stage in the code.

4.1 Importing the Libraries

The project starts with importing Tensorflow and its Keras sub-modules (datasets, layers, and models), together with numpy for array handling and Matplotlib for visualization. Meanwhile, Keras serves ready-made building blocks for the network layers, while the datasets module gives direct access to CIFAR-10 without a manual download as shown in Figure 4.1

```
[2]:
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 4.1 Import libraries

4.2 Loading the CIFAR-10 dataset

The dataset is loaded with datasets.cifar10.load_data() as shown in Figure 4.2.1, which the training set, and test set data is automatically split. CIFAR-10 contains 60,000 colour images grouped into 10 categories which are airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck as shown in Figure 4.2.2.

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Figure 4.2.1 Load datasets

```
[10]:
classes = ['airplane','automobile','bird','cat','deer','dog','frog','horse', 'ship','truck']
```

Figure 4.2.2 datasets classes

Then, inspected the shapes to confirm the structure of the data where the training set hold 50,000 images and the test set hold 10,000 images, each 32 x 32 pixel image with 3 colour channels (RGB), giving the shape (32, 32, 3) per image as shown in Figure 4.2.3.

```
[4]:
X_test.shape

[4]:
(10000, 32, 32, 3)

[5]:
X_train.shape

[5]:
(50000, 32, 32, 3)
```

Figure 4.2.3 Data shape

4.3 Pre-processing the Data

Two preparations steps has been done before training starts:

- Reshaping the labels: Originally the label array is two-dimensional with shape (50000,1) then, they are flattened into a one-dimensional array so that each image maps cleanly to a single class index as shown in Figure 4.3.1.

```
[8]:
y_train = y_train.reshape(-1,)
y_train[:5]

[8]:
array([6, 9, 9, 4, 1], dtype=uint8)
```

Figure 4.3.1 Reshaping labels

- Normalising the pixels: Originally, every pixel value is between integer 0 and 255, then it is divided by 255.0 so that all values fall between 0 and 1. By doing this way, it helps the network train faster and more stably because the optimiser works better with small, uniform input ranges as shown in Figure 4.3.2

```
[14]:
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 4.3.2 Normalising pixels

4.4 Visualising sample images

plot_sample(), uses Matplotlib to display an image at a chosen index together with its class label. This is useful for sanity check where it confirm that the image and their labels line up correctly before any model is build, and it gives a feel for the lower 32 x 32 resolution the model has to work with as shown in Figure 4.4.

```
[12]:
plot_sample(X_train, y_train, 5)
```

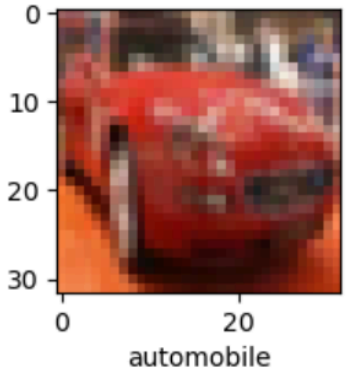

Figure 4.1 plot_sample()

4.5 Building Baseline ANN (for comparison)

Before CNN, a simple Artificial Neural Network (ANN) was built as a baseline. It flattens each image into a long vector and passes it through two large dense layers (3,000 and 1,000 neurons both relu) and a final 10-neuron softmax layer for the class probabilities. It is then compiled with the SGD optimiser and sparse_categorical_crosstropy loss, then trained for 5 epochs as shown in Figure 4.5.1.

```
[15]:
ann = models.Sequential([
    layers.Flatten(input_shape = (32,32,3)),
    layers.Dense(3000, activation = 'relu'),
    layers.Dense(1000, activation = 'relu'),
    layers.Dense(10, activation = 'softmax'),
])

ann.compile(optimizer = 'SGD',
            loss= 'sparse_categorical_crossentropy',
            metrics = ['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

```
C:\Users\ybzah\anaconda3\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWa
rning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the Ten
sorFlow-DirectML plugin.
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 90s 57ms/step - accuracy: 0.3580 - loss: 1.8071
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 89s 57ms/step - accuracy: 0.4278 - loss: 1.6205
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 89s 57ms/step - accuracy: 0.4580 - loss: 1.5374
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 86s 55ms/step - accuracy: 0.4792 - loss: 1.4779
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 88s 57ms/step - accuracy: 0.4991 - loss: 1.4287
[15]:
```

Figure 4.5.1 ANN baseline

The result as shown in Figure 4.5.2 is modest where ANN reaches only 46% accuracy on the test set. This poor result leads to move to a CNN, which is designed specifically for image data because it preserves the spatial relationship between neighboring pixels instead of flattening it away.

[16]:

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification report: \n', classification_report(y_test, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step
classification report:
              precision    recall  f1-score   support

           0       0.34      0.79      0.48      1000
           1       0.64      0.59      0.61      1000
           2       0.40      0.33      0.36      1000
           3       0.36      0.33      0.34      1000
           4       0.35      0.53      0.42      1000
           5       0.49      0.27      0.35      1000
           6       0.50      0.56      0.53      1000
           7       0.60      0.46      0.52      1000
           8       0.77      0.32      0.46      1000
           9       0.67      0.43      0.52      1000

    accuracy                           0.46     10000
   macro avg       0.51      0.46      0.46     10000
weighted avg       0.51      0.46      0.46     10000
```

Figure 4.5.2 ANN classification report

4.6 Building the CNN architecture

The CNN is a sequential model that made if two convolutional blocks followed by a classifier head as shown in Figure 4.6 :

- Convolutional block 1: Conv2D layer with 32 filters of size 3 x 3 and relu activation, followed by a 2 x 2 MaxPooling layer that halves the spatial size and keeps the strongest features.
- Convolutional block 2: Conv2D layer with 64 filters of size 3 x 3 and relu activation, again followed by a 2 x 2 MaxPooling layer.
- Classifier head: Flatten layer that convert the feature maps into vectors, a dense layer of 64 relu neurons learns combinations of those features, and a final Dense layer of 10 neurons with softmax produces the probability for each of the 10 classes.

[19]:

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 4.6 CNN architecture

4.7 Compiling the training CNN

The CNN is compiled with the Adam optimiser (more adaptive compared to SGD), the same sparse_categorical_crossentropy loss, and accuracy as the metric. It is then trained on the training set for 10 epochs using cnn.fit(). During training, the accuracy reaches around 78% of the training data within the first few epochs as the filters learn to recognise useful visual patterns as shown in Figure 4.7.

[20]:

```
cnn.compile(optimizer='adam',
       loss='sparse_categorical_crossentropy',
       metrics=['accuracy'])
```

[21]:

```
cnn.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 29ms/step - accuracy: 0.4619 - loss: 1.4911
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 78s 26ms/step - accuracy: 0.5949 - loss: 1.1502
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 41s 26ms/step - accuracy: 0.6473 - loss: 1.0173
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 82s 27ms/step - accuracy: 0.7801 - loss: 0.6319
```

[21]:

```
<keras.src.callbacks.history.History at 0x17b007fe8a0>
```

Figure 4.7 Compiling and training the CNN

4.8 Evaluating the model

The trained CNN is evaluated with cnn.evaluate(). It achieve test accuracy of approximately 68% which a clear improvement of more than 20 percentage points better than baseline ANN, 46%. This confirm that convolutional approach is far better to image classification as shown in Figure 4.8.

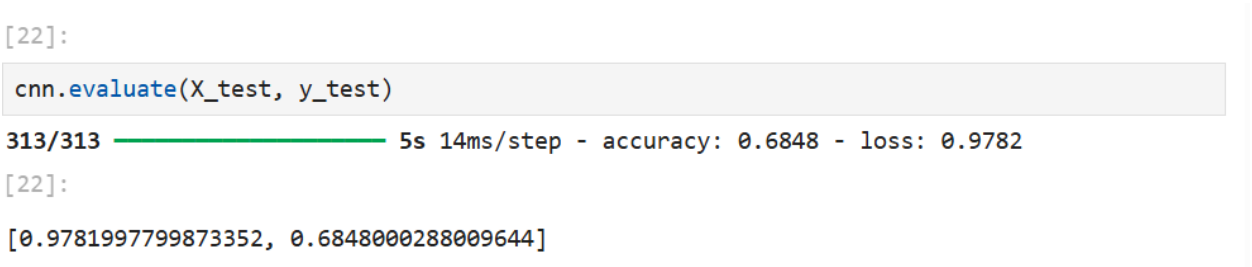

Figure 4.8 Evaluate CNN

4.9 Making predictions

Lastly, cnn.predict() run to the images. The model outputs 10 probabilities per image, and np.argmac() select the class with highest probability as the final prediction as shown in Figure 4.9.

```
[23]:
y_pred = cnn.predict(X_test)
y_pred[:5]

313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step
[23]:

array([[3.4111680e-04, 5.8678870e-05, 8.9790439e-04, 9.3743539e-01,
        1.2473992e-04, 3.2465205e-02, 1.1623671e-03, 6.9418061e-06,
        2.7372940e-02, 1.3462336e-04],
       [1.3466354e-02, 6.6598780e-02, 9.1578577e-06, 5.2734964e-07,
        1.2292918e-08, 6.4152776e-09, 5.5837851e-10, 9.7665764e-10,
        9.1913211e-01, 7.9305004e-04],
       [1.0354129e-01, 2.4486831e-01, 2.5398466e-03, 9.4213886e-03,
        8.2589005e-04, 7.9171045e-04, 9.1058121e-04, 1.0301148e-03,
        4.6599111e-01, 1.7007981e-01],
       [6.8972814e-01, 2.5050601e-03, 5.6365475e-02, 2.3977892e-03,
        5.5616079e-03, 2.5543279e-06, 2.3130963e-04, 7.3207841e-05,
        2.4217933e-01, 9.5560699e-04],
       [4.1546859e-06, 1.6167252e-06, 2.3597052e-02, 5.6897379e-02,
        5.5515766e-01, 3.5682604e-02, 3.2864988e-01, 6.6157077e-06,
        2.9978264e-06, 2.7040494e-08]], dtype=float32)

[24]:
y_classes = [np.argmax(element) for element in y_pred]
y_classes[:5]

[24]:

[3, 8, 8, 0, 4]
```

Figure 4.9 Making predictions

5.0 What is the weakest in the current CNN model?

Although the CNN clearly outperforms the baseline ANN, its test accuracy of around 68% is still modest for CIFAR-10. The single biggest weakness of the current model is its limited ability to generalise where it learns the training data better than it performs on unseen images. Several design choices contribute to this, and they are discussed below:

5.1 Overfitting and the lack of Regularisation

The clearest warning sign is the gap between training and test performance. The model reaches about 78% accuracy on the training set but only 68% on the test. This roughly 10% gap indicates that the model is begin to memorise the training images rather than learning patterns that transfer to new ones. The architecture contains no regularisation at all, there are no dropout layers, no

batch normalization, and no weight regularisation. These are the standard tools for reducing overfitting, and their absence is the most significant weakness of the model.

5.2 No validation set or early stopping

During training, without validation_split or validation_data_argument, the cnn.fit() was called. This means there is no way to monitor performance on unseen data while the model trains. Without validation curve, overfitting cannot be detected as it happens, and useful techniques such as early stopping or learning-rate scheduling cannot be applied. The number of epochs is simply fixed at 10 rather than chosen based on evidence.

5.3 Low capacity architecture

Two convolutional blocks and a single small Dense layer of 64 neurons the network has before the output. Modern image classifiers are usually deeper, with more convolutional layer and more filters, allowing to capture richer and more abstract features. The shallow design here keeps training fast but limits how much the model can actually learn from the data, capping its potential accuracy.

5.4 Weakest on visually similar classes

The model is not equally weak across the categories. As is typical for CIFAR-10, the vehicle classes (airplane, automobile, ship, truck) are easier, while the animal classes that look alike at 32 x 32 resolution are often confused. The per-class result from the baseline highlight where this difficulty is concentrated as shown in Table 5.4.

Table 5.4 Class with its F1 result

Class	**Type**	**Relative difficulty (F1)**
Cat	Animal	0.34 (Lowest)
Dog	Animal	0.35 (Very Low)

Bird	Animal	0.36 (Low)
Deer	Animal	0.42 (Low)
Automobile	Automobile	0.61(High)
Horse/ship/truck	Mixed	0.46-0.52 (Higher)

One of the example is one test image whose true label is frog is misclassified by the CNN as deer. This is because animals share texture, colours and backgrounds, so the model is struggling to tell them apart. This is the weakest are in practice.

5.5 No data Augmentation

The image are fed to the network exactly as they are, with only pixel normalisation applied. CIFAR-10 models benefit greatly from data augmentation because it randomly flips, shifts, rotates or zooms the training images to artificially expand the dataset. Since no augmentation is used, the model sees a limited, fixed view of each image and is more likely to overfit, which limit the accuracy ceiling it can reach.

6.0 How to enhance the CNN model?

The baseline CNN model yielded a test accuracy of 69.04%, suggesting that although spatial features can be learnt from images by the CNN model, there is room for improvement. To increase the strength of the model, several enhancements were made such as Dropout regularisation, additional convolutional layers, Batch Normalization and Data Augmentation. These methods all correct one particular problem with the original model.

6.1 Data Augmentation Setup

In data augmentation, the number of images in the training set is artificially expanded by adding random transformations to the images before a training step, to increase the diversity of the training set. This is done with the help of the Keras's ImageDataGenerator class. The 4 parameters set are:

- **rotation_range=15**: each image in the training set is randomly rotated by up to 15 degrees. This way the model will not learn to always be upright, regardless of the object's orientation.
- **width_shift_range=0.1, height_shift_range=0.1**: images are randomly moved horizontally and/or vertically by up to 10% of the dimension. This helps to model objects to appear slightly at different positions in a frame and increases the robustness of the model to positional variation.
- **horizontal_flip=True**: The images are randomly horizontally flipped. Many real-world objects (animals, vehicles) are valid when flipped and this doubles the number of training samples.

Importantly, augmentation is only done to the training set (X_train). The test set (X_test) does not change, so that the results of the evaluation are not influenced by the use of the transformed test set. The output in the Figure 6.1.1, "Data augmentation ready." shows that the generator has been fitted with the training data distribution.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Apply random transformations to training images only (not test)
datagen = ImageDataGenerator(
    rotation_range=15,       # randomly rotate images by up to 15 degrees
    width_shift_range=0.1,   # shift image horizontally by up to 10%
    height_shift_range=0.1,  # shift image vertically by up to 10%
    horizontal_flip=True     # randomly flip images left-right
)
datagen.fit(X_train)
print("Data augmentation ready.")
```

```
Data augmentation ready.
```

Figure 6.1.1 *Data Augmentation Setup*

6.2 Enhanced CNN Architecture

The enhanced CNN model (cnn_new) introduces three major structural improvements over the original CNN. The architecture is organised into three convolutional blocks followed by a dense classification head.

1. **Block 1 — 32 filters (Figure 6.2.1):** Two consecutive Conv2D layers with 32 filters each, both using padding='same' so the spatial dimensions (32×32) are preserved after convolution. Each convolution is immediately follo by BatchNormalization, which normalises the activation values to have a mean near zero and unit variance. This stabilises training and allows higher learning rates. A MaxPooling2D layer then halves the spatial size to 16×16, and Dropout(0.25) randomly deactivates 25% of feature maps to prevent overfitting.
2. **Block 2 — 64 filters (Figure 6.2.1):** Same structure as Block 1 but with 64 filters, allowing the model to detect more complex and abstract patterns (e.g. combinations of edges that form shapes). The feature map size is reduced to 8×8 after pooling.

```python
from tensorflow.keras.layers import BatchNormalization, Dropout

cnn_new = models.Sequential([

    # Block 1 – 32 filters (two conv layers back to back)
    layers.Conv2D(32, (3,3), padding='same', activation='relu', input_shape=(32,32,3)),
    BatchNormalization(),
    layers.Conv2D(32, (3,3), padding='same', activation='relu'),
    BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    Dropout(0.25),

    # Block 2 – 64 filters
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    BatchNormalization(),
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    Dropout(0.25),
```

Figure 6.2.1 *Enhanced CNN Architecture (Block 1 - 2)*

3. ***Block 3 in*** **Figure 6.2.2** ***— 128 filters (extra layer vs original CNN):*** *A single Conv2D layer with 128 filters extracts the highest-level features (e.g. object parts). This block does not exist in the original CNN and represents the primary architectural expansion. MaxPooling reduces the map to 4×4.*
4. ***Dense head*** **(Figure 6.2.2)*:*** *The feature maps are flattened into a vector of 2,048 values (4×4×128), then passed through a Dense(256) layer with ReLU activation. BatchNormalization and a higher Dropout(0.5) are applied before the final Dense(10) output layer with softmax, which produces probability scores for all 10 CIFAR-10 classes.*

```
    # Block 3 – 128 filters (extra layer vs original CNN)
    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    Dropout(0.25),

    # Dense head
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

cnn_new.compile(optimizer='adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])

cnn_new.summary()
```

Figure 6.2.2 *Enhanced CNN Architecture (Block 3 & Dense head)*

The summary in **Figure 6.2.3 - 6.2.5** confirms the full layer stack and shows a total of 668,842 trainable parameters (2.55 MB). This is significantly more than the original CNN's ~122,570 parameters, reflecting the deeper and wider architecture. The Non-trainable params: 1,152 correspond to the BatchNormalization gamma and beta parameters that are frozen during inference.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_3 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_4 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_5 (Conv2D)	(None, 16, 16, 64)	36,928

Figure 6.2.3 *Output of Enhanced CNN Architecture*

batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_6 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_4 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_5 (Dense)	(None, 256)	524,544
batch_normalization_5 (BatchNormalization)	(None, 256)	1,024
dropout_3 (Dropout)	(None, 256)	0

Figure 6.2.4 *Output of Enhanced CNN Architecture*

dense_6 (Dense)	(None, 10)	2,570

Total params: 668,842 (2.55 MB)
Trainable params: 667,690 (2.55 MB)
Non-trainable params: 1,152 (4.50 KB)

Figure 6.2.5 *Output of Enhanced CNN Architecture*

6.3 Train Enhanced CNN (Training Log)

The enhanced model in **Figure 6.3** is trained using cnn_new.fit() with datagen.flow() as the data source. Instead of passing X_train directly, datagen.flow() generates augmented batches of 64 images on the fly during each epoch. This means no two epochs ever see the identical set of images, which strongly reduces overfitting.

Training ran for 10 epochs, consistent with the original CNN for a fair comparison. The log shows both training accuracy and val_accuracy (validation accuracy on X_test) per epoch:

- **Epoch 1:** train_acc = 42.37%, val_acc = 54.90% — the model starts below the original CNN's epoch 1 (48.52%). This is expected: augmented images are harder, so the model learns more slowly at first.
- **Epoch 5:** train_acc = 68.84%, val_acc = 72.39% — by mid-training the validation accuracy already surpasses the original CNN's final test accuracy (69.04%).
- **Epoch 10:** train_acc = 74.85%, val_acc = 78.48%. The gap between training and validation accuracy is small (≈3.6%), indicating that overfitting has been well controlled through Dropout and data augmentation.

```
history_new = cnn_new.fit(
    datagen.flow(X_train, y_train, batch_size=64),
    epochs=10,
    validation_data=(X_test, y_test),
    verbose=1
)

Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 435s 548ms/step - accuracy: 0.4237 - loss: 1.6978 - val_accuracy: 0.5490 - val_loss: 1.2585
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 415s 531ms/step - accuracy: 0.5699 - loss: 1.2009 - val_accuracy: 0.6263 - val_loss: 1.0596
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 400s 512ms/step - accuracy: 0.6308 - loss: 1.0418 - val_accuracy: 0.6710 - val_loss: 0.9945
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 386s 494ms/step - accuracy: 0.6681 - loss: 0.9498 - val_accuracy: 0.6859 - val_loss: 0.8890
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 385s 493ms/step - accuracy: 0.6884 - loss: 0.8879 - val_accuracy: 0.7239 - val_loss: 0.8008
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 413s 528ms/step - accuracy: 0.7078 - loss: 0.8338 - val_accuracy: 0.7204 - val_loss: 0.8193
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 422s 540ms/step - accuracy: 0.7181 - loss: 0.8077 - val_accuracy: 0.7224 - val_loss: 0.8129
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 394s 504ms/step - accuracy: 0.7315 - loss: 0.7716 - val_accuracy: 0.7580 - val_loss: 0.7231
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 393s 502ms/step - accuracy: 0.7418 - loss: 0.7454 - val_accuracy: 0.7564 - val_loss: 0.7099
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 390s 498ms/step - accuracy: 0.7485 - loss: 0.7247 - val_accuracy: 0.7848 - val_loss: 0.6348
```

Figure 6.3 *Train Enhanced CNN*

6.4 Evaluate Enhanced CNN (Test Accuracy)

After training, the enhanced model **(Figure 6.4)** is evaluated against the test set using cnn_new.evaluate(X_test, y_test). The output shows:

Enhanced CNN — Test Accuracy : 78.48% (up from 69.04% for the original CNN)

Enhanced CNN — Test Loss : 0.6348 (down from 0.9700 for the original CNN)

The improvement of +9.44 percentage points in test accuracy demonstrates that the combined enhancements — deeper architecture, Batch Normalization, Dropout regularisation, and data augmentation — are effective. The lower loss value also confirms that the enhanced model's probability estimates are better calibrated across all 10 classes.

```
cnn_new_loss, cnn_new_acc = cnn_new.evaluate(X_test, y_test, verbose=1)
print(f"\nEnhanced CNN — Test Accuracy : {cnn_new_acc*100:.2f}%")
print(f"Enhanced CNN — Test Loss     : {cnn_new_loss:.4f}")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 16s 50ms/step - accuracy: 0.7848 - loss: 0.6348

Enhanced CNN — Test Accuracy : 78.48%
Enhanced CNN — Test Loss     : 0.6348
```

Figure 6.4 *Evaluate Enhanced CNN*

7.0 What is the evaluation based on CNN and the new CNN model?

This section presents a direct comparison between the three models — ANN (baseline), the original CNN, and the Enhanced CNN — using both a numerical table and a bar chart.

7.1 Comparison Table

The comparison table in **Figure 7.1.1** presents the final test accuracy of all three models alongside the improvement relative to the ANN baseline:

ANN (Baseline): 46.00% — The ANN flattens each 32×32×3 image into a 3,072-element vector, losing all spatial information. It cannot exploit the local structure of pixel neighbourhoods, resulting in the lowest accuracy.

CNN: 70.36% (+24.4% vs ANN) — By using convolutional filters, the CNN preserves and exploits spatial relationships between pixels. The jump of over 24 percentage points over the ANN confirms the fundamental advantage of CNNs for image classification tasks.

Enhanced CNN: 78.48% (+32.5% vs ANN) — The additional convolutional block, Batch Normalization, Dropout, and data augmentation push accuracy to 78.48%, a further +8.12% over the plain CNN. This demonstrates that architectural depth and regularisation techniques have a measurable positive impact.

Model	Test Accuracy	vs ANN
ANN (Baseline)	46.00%	–
CNN	70.36%	+24.4%
Enhanced CNN	78.48%	+32.5%

Figure 7.1.1 *Comparison Table*

7.2 Bar Chart

The bar chart in **Figure 7.2.1** provides a visual summary of the three models' test accuracies. The dashed horizontal line marks the ANN baseline at 46%, making it easy to

see how much each CNN variant improves upon the non-convolutional approach. The green bar (Enhanced CNN at 78.48%) clearly stands tallest, illustrating the cumulative benefit of all enhancements applied. The chart was generated using Matplotlib with labelled percentage values displayed directly above each bar for readability.

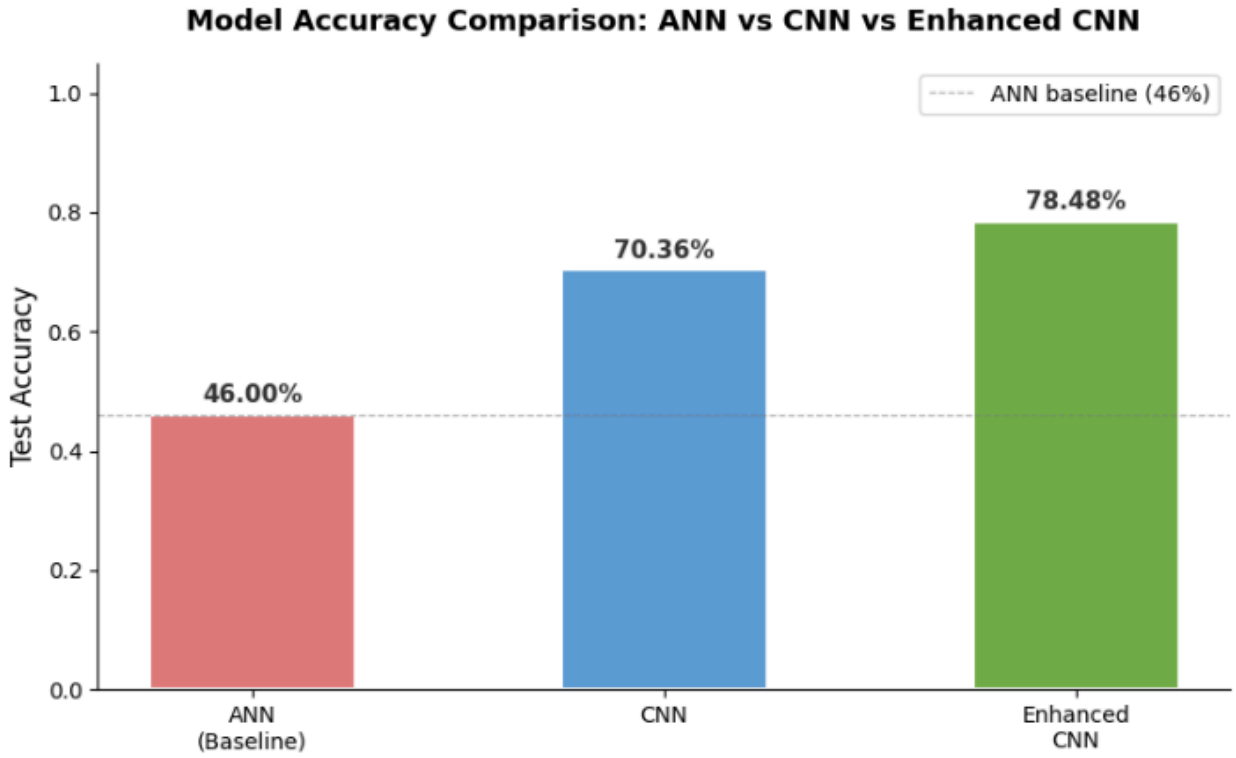

Figure 7.2.1 *Bar Chart*

8.0 Results and Discussion

The results of this tutorial demonstrate a clear progression in model performance as the architecture and training strategy become more sophisticated. Starting from a basic Artificial Neural Network (ANN) which treats images as flat vectors, moving to a Convolutional Neural Network (CNN) that preserves spatial structure, and finally to an Enhanced CNN with deeper layers and regularisation, each step produced a meaningful improvement in test accuracy on the CIFAR-10 dataset.

The ANN achieved only 46.00% accuracy, confirming that fully-connected networks are inherently unsuitable for image classification at scale. When pixels are flattened, the model cannot distinguish between local patterns such as edges, corners, or textures — all of which are critical for distinguishing between CIFAR-10 categories such as "cat" versus "deer" or "automobile" versus "truck".

The original CNN improved accuracy to 70.36% by applying convolutional filters across the spatial dimensions of each image. This allowed the model to detect low-level features (edges, gradients) in early layers and progressively more complex features (shapes, object parts) in deeper layers. However, with only two convolutional blocks and no regularisation, the model showed signs of overfitting — training accuracy reached 80.41% while test accuracy was 70.36%, a gap of approximately 10 percentage points.

The Enhanced CNN addressed these limitations through four targeted improvements. First, a third convolutional block with 128 filters was added to capture higher-level abstract features. Second, Batch Normalization was applied after every convolutional layer, which accelerated learning and reduced sensitivity to weight initialisation. Third, Dropout (0.25 in convolutional blocks, 0.5 before the output layer) penalised over-reliance on any single neuron pathway, directly reducing overfitting. Fourth, data augmentation ensured the model was exposed to a wider variety of image transformations during training, improving generalisation to unseen data. The resulting test accuracy of 78.48% — with a train-to-test gap of only 3.6% — confirms that both performance and generalisation improved substantially.

In summary, these results validate the theoretical advantages of CNNs over ANNs for image data, and demonstrate that depth, normalisation, and regularisation are effective strategies for enhancing CNN performance. Further improvements could be achieved by training for more epochs, using a learning rate scheduler, or incorporating residual connections (as in ResNet-style architectures).

9.0 Reflection

NURUL ADRIANA BINTI KAMAL JEFRI (A23CS0258)

Working on the enhancement and evaluation parts of this tutorial enabled a much better understanding of the direct impact of architectural decisions on the generalisation power of the model. Initially it was thought that the more layers the better, but it was found that a deeper network can overfit, or memorise the training data, without the use of regularisation techniques like Dropout and Batch Normalization. The most difficult piece was the inability to understand how the improvements in the training accuracy of the enhanced CNN (74.85%) led to a decrease in the original CNN's training accuracy (80.41%), even though its test accuracy was higher (78.48% vs 69.04%). It was subsequently found that this was a symptom of improved training: it was making the task more difficult through each epoch with the intent to make the model more general rather than overfit. Creating the side-by-side comparison table and bar chart also underscored the importance of conveying the performance of the model clearly and objectively, a key aspect of data engineering practice. In the future, the additional training of the enhanced model for several more epochs with a learning rate scheduler, as well as a trying out of residual connections, as they use in ResNet might be worthwhile, potentially reaching over 85% accuracy on CIFAR-10 without spending much more time for training.

YASMIN BATRISYIA BINTI ZAHIRUDDIN (A23CS0201)

By doing this tutorial has given me a much clearer understanding of how image classifiers are actually built. The part I found most interesting was seeing why a CNN works better than a plain ANN. When the baseline ANN only reached around 46% accuracy, I understand why flattening an image throws away important information, the network loses the spatial relationship between neighbouring pixels. Then, watching the CNN reach 68% on the same data by using convolution and pooling to keep that spatial structure has made the theory make sense. Small steps like normalising the pixels and reshaping the labels also taught me that data preparation is just important as the model itself. Moreover, I initially thought that higher accuracy simply meant a better model, but looking closely I realised the gap between the training accuracy (~78%) and the test accuracy (~68%) was a sign of overfitting. The reason accuracy was pleated because of no regularisation, no data augmentation, and no validation set to monitor

it while training. I also found it interesting that the model struggled most with visually similar animals which made sense once I considered how little detail a 32 x 32 image actually contains. Overall, the tutorial has taught me how and why the model behaves like that which has increased my understanding.

SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

Implementing the practical steps of this tutorial provided a grounded understanding of end-to-end CNN training that theory alone cannot replicate. The most revealing phase involved plotting the training and validation accuracy curves. Observing the widening gap, where training accuracy climbed while validation stagnated, offered a concrete demonstration of overfitting. Minor pipeline adjustments, such as normalizing pixel values and optimizing batch sizes, significantly improved convergence smoothness, reinforcing the principle that data preparation holds equal weight to network architecture. An initial analysis revealed a discrepancy where overall accuracy appeared satisfactory, yet performance plummeted on specific classes. Examining per-class metrics isolated the issue, demonstrating that visually similar categories like cats and dogs accounted for the vast majority of misclassifications. Future iterations would benefit from exploring transfer learning via pretrained models like ResNet or EfficientNet, as leveraging features derived from larger datasets can substantially elevate accuracy without accelerating training times.

References

Ioffe, S., & Szegedy, C. (2015, June 1). *Batch normalization: Accelerating deep network training by reducing internal covariate shift*. PMLR. https://proceedings.mlr.press/v37/ioffe15.html

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, *60*(6), 84–90. https://doi.org/10.1145/3065386

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324. https://doi.org/10.1109/5.726791

Simplilearn. (2022). Image Classification Using CNN | Deep Learning Projects. YouTube. https://www.youtube.com/watch?v=Rmtr9SY-4VQ

Srivastava, N., Hinton, G. E., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(1),1929–1958. https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf